

ANTEPROYECTO

Aplicación Web para la Gestión de Incidencias IT

Alumno	Ignacio Martínez Torres
Ciclo	2.º Desarrollo de Aplicaciones Web (DAW)
Módulo	Proyecto Integrado
Centro	IES / Centro de Formación Profesional
Curso académico	2025 – 2026
Versión del documento	v2.0 — Revisión y ampliación final

1. Introducción y Estudio del Problema

1.1. Descripción del Proyecto

IMTracker es una aplicación web completa para la gestión interna de incidencias del departamento de IT de una empresa. La plataforma centraliza, gestiona y da seguimiento a los problemas técnicos reportados por los distintos departamentos de la organización (Ventas, Compras) y gestionados por el personal técnico de IT.

El sistema implementa un ciclo de vida completo del ticket, desde su creación por un usuario estándar hasta su resolución y cierre por parte del equipo técnico, con trazabilidad total de todas las acciones realizadas. Adicionalmente, incorpora inteligencia artificial local para el análisis automático de incidencias, un asistente conversacional IT, soporte bilingüe completo y una infraestructura de despliegue en la nube con contenerización Docker y pipeline CI/CD.

1.2. Problemática Actual

La gestión de incidencias en la mayoría de las pymes se realiza de forma informal: correos electrónicos dispersos, mensajes de WhatsApp o comunicación telefónica. Esta situación genera consecuencias directas sobre la productividad del departamento IT:

- Pérdida de información crítica sobre el estado y contexto de las tareas.
- Duplicidad de esfuerzos al no existir un registro centralizado.
- Ausencia total de métricas sobre la productividad y tiempos de resolución del departamento IT.
- Imposibilidad de priorizar correctamente las tareas urgentes frente a las de baja criticidad.
- Falta de trazabilidad y auditoría de las acciones realizadas por cada técnico.
- Dificultad para detectar patrones recurrentes de incidencias que podrían resolverse de forma preventiva.

IMTracker aborda todos estos problemas proporcionando una herramienta unificada, segura y accesible desde cualquier dispositivo, adaptada a la jerarquía real de los equipos IT.

2. Objetivos

2.1. Objetivo General

Desarrollar una aplicación web robusta y escalable mediante el framework Django (Python) que permita gestionar el ciclo de vida completo de una incidencia IT, priorizando la seguridad, la trazabilidad y la experiencia de usuario, e incorporando capacidades de inteligencia artificial local para el análisis automático de tickets.

2.2. Objetivos Específicos

Gestión de usuarios y roles

- Autenticación segura con sesiones protegidas (Login / Logout / recuperación de contraseña).
- Roles diferenciados con permisos estrictos por fila: Usuario (Ventas/Compras), Técnico IT y Manager IT.
- Cambio de contraseña con validación mediante el motor nativo de Django.
- Importación masiva de usuarios desde CSV con asignación automática de departamento y credenciales.

Gestión de incidencias (CRUD completo)

- Creación de tickets mediante modales y AJAX, sin recarga de página.
- Visualización en tabla dinámica (DataTables) con filtros en tiempo real por ID, título y descripción.
- Edición, asignación y borrado lógico (soft-delete con campo oculta) de incidencias.
- Insercción de capturas de pantalla al ticket para facilitar el diagnóstico técnico.
- Descarga individual de tickets en formato PDF.

Lógica de negocio avanzada

- Filtrado de visibilidad por departamento: los usuarios solo ven los tickets de su propio departamento; el equipo IT tiene visibilidad global.
- Sistema de estados con transiciones controladas: Pendiente, En curso, Faltan datos, Testing, Finalizada.
- Restricciones de borrado: solo incidencias propias en estado "Pendiente" para usuarios no-IT.
- Filtros combinados: selección múltiple simultánea de estados y técnicos asignados.
- Sistema de observaciones: comentarios adicionales vinculados a cada ticket por cualquier usuario con acceso.

Inteligencia artificial e integración

- Análisis automático de cada ticket al crearse: sugerencia de prioridad y resumen técnico mediante modelo LLM local (Ollama).
- Asistente conversacional IT (Chatbot): chatbot inteligente que analiza el historial de incidencias para sugerir soluciones y responder dudas en tiempo real, sin enviar datos a servicios externos.

- Privacidad garantizada: toda la inferencia de IA se ejecuta localmente, sin llamadas a APIs de terceros.

Infraestructura, DevOps y seguridad

- Contenerización completa con Docker (aplicación + base de datos aisladas).
- Pipeline CI/CD con GitHub Actions para despliegue automatizado en AWS EC2 tras cada push al repositorio.
- Seguridad SSL mediante Nginx y certificados Let's Encrypt para cifrado de extremo a extremo.
- Sistema de logs de auditoría: registro de eventos críticos del servidor en ficheros físicos para trazabilidad de seguridad.

Internacionalización y experiencia de usuario

- Interfaz totalmente traducida al Español e Inglés, incluyendo selector de idioma en la pantalla de login.
- Traducciones dinámicas en JavaScript para componentes cargados por AJAX.
- Motor de notificaciones UX: modales personalizados con animaciones que sustituyen a las alertas nativas del navegador.
- Diseño 100 % responsivo con CSS nativo (Flexbox / Grid), arquitectura BEM.
- Buzón de sugerencias con notificación automática por email (SMTP Gmail) al equipo IT.

3. Planteamiento y Justificación de la Solución

3.1. Solución Elegida

IMTracker se desarrolla como una aplicación web basada en el patrón MVT (Model-View-Template) propio de Django, con una arquitectura modular y desacoplada que facilita el mantenimiento y la escalabilidad futura.

3.2. Justificación Tecnológica

Tecnología	Categoría	Justificación
Django 5.x	Backend / Framework	Desarrollo rápido, seguridad integrada (CSRF, XSS, SQLi), ORM potente, sistema de autenticación nativo y soporte i18n completo.
PostgreSQL	Base de datos	SGBD relacional robusto, superior a SQLite en concurrencia y para simular entornos de producción reales.
Vanilla CSS + BEM	Frontend / Estilos	Sin dependencias externas de CSS. Arquitectura BEM que garantiza escalabilidad, legibilidad y rendimiento máximo del frontend.
Vanilla JS (modular)	Frontend / Lógica	Arquitectura JavaScript estructurada y modular sin frameworks pesados, demostrando dominio del lenguaje base.
jQuery + DataTables	Frontend / Tablas	Soporte AJAX y renderizado dinámico de tablas con filtros, paginación y búsqueda en tiempo real.
Ollama (LLM local)	Inteligencia Artificial	Inferencia de modelos de lenguaje totalmente local, sin dependencia de APIs externas ni envío de datos sensibles a terceros. Garantía de privacidad RGPD.
Docker	Contenerización	Entornos reproducibles e independientes del sistema operativo del servidor. Facilita el despliegue y el escalado horizontal.
GitHub Actions	CI/CD	Automatización de pruebas y despliegue continuo en AWS EC2 tras cada commit, garantizando entregas ágiles y seguras.
AWS EC2 + Nginx	Infraestructura cloud	Servidor cloud de alta disponibilidad con proxy inverso Nginx y SSL (Let's Encrypt) para cifrado de extremo a extremo.
WeasyPrint	Generación de PDFs	Librería Python para renderizar HTML/CSS a PDF, generando tickets e informes de resolución listos para archivar.
SMTP (Gmail)	Comunicaciones	Envío de notificaciones reales por correo

		electrónico a técnicos IT y flujo seguro de recuperación de contraseña.
--	--	---

4. Modelado de la Solución

4.1. Arquitectura del Sistema

La aplicación sigue el patrón MVT de Django. Las peticiones HTTP son gestionadas por el enrutador de URLs, que las delega a las vistas correspondientes. Las vistas interactúan con los modelos (ORM de Django sobre PostgreSQL) y renderizan las plantillas HTML. Las operaciones asíncronas se realizan mediante AJAX desde el cliente. El módulo de IA (Ollama) se integra como servicio local al que el backend consulta de forma síncrona al crear cada ticket.

4.2. Modelo de Datos Principal

El esquema de la base de datos se articula en torno a tres entidades principales:

Entidad	Campos clave	Descripción
UsuarioPersonalizado	id, username, first_name, last_name, email, departamento	Extiende AbstractUser de Django añadiendo el campo departamento con opciones: Ventas, Compras, IT, Manager IT.
Incidencia	id, titulo, descripcion, prioridad, estado, creador_id (FK), asignado_a_id (FK), imagen, oculta, resumen_ia, prioridad_sugerida_ia, fechas	Entidad central del sistema. Relaciona al creador y al técnico asignado. Incluye campos de IA para el resumen y la prioridad sugerida. El borrado es lógico mediante el campo oculta.
Observacion	id, incidencia_id (FK), autor_id (FK), texto, fecha	Permite añadir comentarios adicionales a una incidencia. Cualquier usuario con acceso al ticket puede escribir observaciones.

4.3. Recursos del Proyecto

Recursos Humanos

- Desarrollador: 1 persona (alumno), encargada del análisis, diseño, implementación Full Stack y pruebas.
- Tutor/a del módulo: supervisión periódica y validación de hitos.

Recursos Hardware

- Ordenador personal para el desarrollo local.
- Instancia AWS EC2 (t2.micro / t3.small) para el despliegue en producción.

Recursos Software

- IDE: Visual Studio Code con extensiones Python, Django y Docker.
- Lenguajes: Python 3.x, JavaScript ES6+, HTML5, CSS3.
- Frameworks y librerías: Django 5.x, jQuery, DataTables, WeasyPrint, Nager.Date API.
- SGBD: PostgreSQL.
- IA: Ollama con modelo LLM local.
- Control de versiones: Git + GitHub (repositorio privado).
- Contenerización: Docker + Docker Compose.
- CI/CD: GitHub Actions.
- Servidor web: Nginx con SSL (Let's Encrypt).

5. Planificación Temporal

El desarrollo se estructura en fases secuenciales, siguiendo una metodología iterativa e incremental. Cada fase entrega funcionalidad verificable antes de comenzar la siguiente.

#	Fase	Contenido	Estado
0	Infraestructura y entorno	Configuración de entorno virtual, Django, PostgreSQL, Git y estructura de carpetas del proyecto.	Completada
1	Autenticación	Login, logout, control de sesiones, redireccionamientos por rol, cambio y recuperación de contraseña.	Completada
2	Dashboard y listado	DataTables con paginación, buscadores y visualización de campos básicos (ID, título, estado, prioridad, asignado).	Completada
3	Creación de incidencias	Formularios en modales con AJAX, subida de imágenes y validaciones de servidor y cliente.	Completada
4	Lógica de negocio y permisos	Visibilidad por departamento, sistema de estados con colores, restricciones por rol, sistema de observaciones.	Completada
5	Detalle y edición	Vista modal con info completa, fechas, imágenes adjuntas y edición restringida por autoría.	Completada
6	Acciones de gestión	Asignación, cambio de estado, borrado lógico (soft-delete), exportación PDF y CSV.	Completada
7	Filtros avanzados	Filtros combinados de selección múltiple por estado y técnico asignado de forma simultánea.	Completada
8	UX y maquetación final	Arquitectura CSS/BEM, diseño responsivo, motor de modales UX y mejora de experiencia de usuario global.	Completada
9	Inteligencia Artificial	Integración de Ollama para análisis automático de tickets (sugerencia de prioridad + resumen técnico) y Chatbot IT conversacional.	Completada
10	Internacionalización	Soporte bilingüe Español/Inglés con selector de idioma en login y traducciones dinámicas en JavaScript.	Completada
11	DevOps y despliegue	Contenerización Docker, pipeline CI/CD con GitHub Actions, despliegue en AWS EC2, configuración Nginx + SSL.	Completada
12	Comunicaciones y auditoría	SMTP Gmail para notificaciones y recuperación de contraseña. Logs de auditoría en ficheros físicos. Buzón de sugerencias.	Completada

6. Ampliaciones Implementadas Respecto al Anteproyecto Inicial

Tras la revisión de los criterios de calificación del Proyecto Integrado y la validación del enfoque con el profesorado, el alcance del proyecto se amplió considerablemente. A continuación se detallan todas las funcionalidades que superan la propuesta original:

6.1. Inteligencia Artificial Local (Ollama)

En lugar de la herramienta n8n planteada inicialmente, se optó por integrar directamente Ollama como motor de IA local, garantizando la privacidad de los datos y eliminando la dependencia de servicios externos:

- **Análisis de tickets:** Análisis automático al crear un ticket: el modelo LLM analiza la descripción y genera un resumen técnico y una sugerencia de prioridad almacenados en los campos `resumen_ia` y `prioridad_sugerida_ia` del modelo Incidencia.
- **Asistente IT:** Chatbot IT conversacional: asistente inteligente integrado en el panel lateral que consulta el historial de incidencias de la empresa para sugerir soluciones y resolver dudas técnicas en tiempo real.

6.2. Infraestructura de Producción (DevOps)

- **Docker:** Contenedores Docker independientes para la aplicación Django y la base de datos PostgreSQL.
- **CI/CD:** Pipeline CI/CD completamente automatizado con GitHub Actions: cada push al repositorio lanza las pruebas y, si pasan, despliega la nueva versión en la instancia AWS EC2.
- **Nginx + SSL:** Proxy inverso Nginx con certificado SSL/TLS (Let's Encrypt) para cifrado de extremo a extremo en producción.
- **Auditoría:** Logs de auditoría: sistema de trazabilidad que registra en ficheros físicos cada evento crítico del servidor (accesos, errores, acciones administrativas).

6.3. Funcionalidades Backend Adicionales

- **Exportación PDF:** Generación de tickets individuales en PDF mediante WeasyPrint, descargables por el usuario creador y por el equipo IT.
- **Exportación CSV:** Exportación de listados de incidencias en formato CSV para auditoría y análisis de métricas del departamento.
- **Importación CSV:** Importación masiva de usuarios desde archivos CSV, con creación automática de cuentas, asignación de departamento y contraseña por defecto segura.
- **Sugerencias:** Buzón de sugerencias con notificación automática por email al equipo IT al recibir una nueva propuesta de mejora.
- **Reset de contraseña:** Recuperación de contraseña mediante enlace único y seguro enviado al correo electrónico del usuario.
- **API Festivos:** Integración con la API pública Nager.Date para la consulta de festivos nacionales, facilitando la optimización de tiempos de respuesta técnica.

6.4. Frontend y Experiencia de Usuario

El apartado de maquetación y UX se amplió significativamente respecto a la propuesta inicial:

- **CSS/BEM:** Arquitectura CSS 100 % propia con metodología BEM (Block, Element, Modifier), sin dependencia de frameworks externos como Bootstrap o Tailwind. Se demuestra dominio del CSS nativo.
- **Responsive:** Diseño completamente responsivo mediante CSS Grid y Flexbox nativos para adaptabilidad en cualquier dispositivo.
- **Modales UX:** Motor de notificaciones UX personalizado: modales con animaciones fluidas que sustituyen completamente a las alertas nativas del navegador (alert, confirm), mejorando la coherencia visual.
- **Internacionalización:** Soporte bilingüe completo (Español / Inglés) con selector de idioma en la pantalla de login e i18n nativo de Django. Las cadenas cargadas dinámicamente por AJAX también están traducidas mediante variables JavaScript inyectadas por Django.
- **Observaciones:** Sistema de observaciones: interfaz para añadir comentarios adicionales a cualquier incidencia, con autoría y fecha registradas.

7. Resultado Final y Estado del Proyecto

El proyecto se encuentra en estado estable y operativo. Todas las fases planificadas han sido completadas, incluyendo la totalidad de las ampliaciones descritas en el apartado anterior. A continuación se resume el conjunto de funcionalidades entregadas:

Módulo / Funcionalidad	Estado
Autenticación, roles y permisos por departamento	Implementado y en producción
CRUD completo de incidencias con AJAX	Implementado y en producción
Sistema de estados y filtros avanzados combinados	Implementado y en producción
Sistema de observaciones y adjuntos de imagen	Implementado y en producción
IA local: análisis automático de tickets (Ollama)	Implementado y en producción
Chatbot IT conversacional con contexto de incidencias	Implementado y en producción
Importación / exportación masiva CSV	Implementado y en producción
Exportación de tickets individuales a PDF	Implementado y en producción
Notificaciones SMTP y recuperación de contraseña	Implementado y en producción
Logs de auditoría en ficheros físicos	Implementado y en producción
Soporte bilingüe Español / Inglés (i18n completo)	Implementado y en producción
Motor de modales UX y notificaciones personalizadas	Implementado y en producción
Buzón de sugerencias con notificación a IT	Implementado y en producción
API Nager.Date para festivos nacionales	Implementado y en producción
Docker + CI/CD GitHub Actions + AWS EC2	Implementado y en producción
Nginx + SSL (Let's Encrypt) en producción	Implementado y en producción

La documentación del proyecto incluye el presente anteproyecto, el Manual de Usuario (USER_MANUAL.md) con cobertura exhaustiva de todas las funcionalidades, y el README técnico (README.md) con instrucciones de instalación, despliegue y descripción del stack tecnológico completo.